COLLEGE MINI-PROJECT REPORT

Smart Campus Navigation System

A C11 terminal application demonstrating graph-based campus walking routes

Student Name	[STUDENT NAME]
Roll Number	[ROLL NUMBER]
Class / Division	[CLASS / DIVISION]
Department / College	[DEPARTMENT / COLLEGE NAME]
Project Guide	[FACULTY GUIDE NAME]
Academic Year	[ACADEMIC YEAR]

Submitted in partial fulfilment of the requirements for
[COURSE / SUBJECT]

Submission date: [DATE]

CERTIFICATE (TEMPLATE)

This is to certify that the mini-project report titled **"Smart Campus Navigation System"** is submitted by **[STUDENT NAME]**, Roll No. **[ROLL NUMBER]**, of **[CLASS / DIVISION]**, for the academic year **[ACADEMIC YEAR]**. It is presented for academic evaluation in accordance with the requirements of **[COURSE / DEPARTMENT]**.

This is a blank certificate template. It does not represent college approval, an evaluation result, or a signature.

_____
Project Guide Signature
Name: [FACULTY GUIDE]

_____
Head of Department
Name: [AUTHORIZED SIGNATORY]

Date: _____

Institution Seal (if authorized)

ACKNOWLEDGEMENT

I would like to express my gratitude to **[FACULTY GUIDE NAME]** for guidance and feedback during this mini-project. I also thank **[DEPARTMENT / COLLEGE NAME]** for providing the academic setting in which the work was completed. I acknowledge the C language and compiler documentation referenced in this report and the source materials included with the project.

This acknowledgement is a template. Replace bracketed text with accurate details and retain only statements that reflect assistance actually received.

ABSTRACT

The Smart Campus Navigation System is a standalone C11 console application that models campus locations as vertices and walking paths as positive, undirected distance-weighted edges. It stores location records in a dynamically allocated singly linked list and the network in a fixed-capacity adjacency-matrix graph. The program provides location insertion, display and lookup; path insertion and listing; minimum-distance route computation using Dijkstra's algorithm; reachability traversal using breadth-first search (BFS) and a circular queue; graph connectivity checking; and an optional fictional sample map. The sample map contains five locations and seven paths and is explicitly identified by the program as fictional. A fresh GCC build with warning flags completed without diagnostics. The separate module test driver passed. CLI regressions passed when the temporary test expectation for duplicate-ID wording was aligned with the actual application message; the unchanged test script has that one stale expected string. The project does not connect to GPS, live maps, online services, or real-time campus data.

Keywords: C11, linked list, weighted graph, adjacency matrix, Dijkstra, BFS, circular queue, campus navigation.

TABLE OF CONTENTS

CERTIFICATE (TEMPLATE) 2

ACKNOWLEDGEMENT 3

ABSTRACT 4

1. Introduction and Problem Statement 6

2. Project Objectives and Scope 7

3. Software and Hardware Requirements 8

4. Existing System and Proposed Approach 9

5. Data Structures Used 10

6. Algorithms Implemented 11

7. System Architecture and Program Flow 12

8. Module-wise Explanation and Implementation Details 13

9. Input, Output, Compilation, and Execution 14

10. Testing Methodology and Results 15

11. Program Output Screenshots 16

12. Limitations and Future Enhancements 21

13. Conclusion 22

14. References 23

1. Introduction and Problem Statement

A campus can be represented as named places connected by walkable paths. A user who knows a source and destination may need to compare possible routes, determine which places are reachable, or inspect whether the modeled network is connected. This mini-project demonstrates those operations with a small, editable in-memory graph and a text menu.

The software is an educational graph model, not an operational navigation product. It does not acquire coordinates, distances, or routes from a physical campus. The user enters path lengths in metres, and the optional demonstration data is invented for development and testing.

Problem statement

Given a finite set of campus locations and positive distances between selected pairs, represent the network in memory and provide menu operations to manage locations and paths, find a least-total-distance route, enumerate reachable locations, and determine connectivity. Handle invalid input and invalid graph operations without accepting malformed records.

2. Project Objectives and Scope

Objectives

- Represent named locations using a singly linked list with unique positive IDs and trimmed names unique without regard to ASCII letter case.
- Represent walking links and distances in an undirected graph with an adjacency matrix and a 100-vertex capacity.
- Calculate a minimum-distance route using Dijkstra's algorithm and return its route and total distance in metres.
- Traverse reachable vertices with BFS using a custom circular queue, and check graph connectivity.
- Use a menu with validated integer input and test the list, graph, queue, and command-line behavior.

Scope

The application operates locally in a terminal. Location and edge data exist only for the current run. A user may add locations and undirected paths, display locations and paths, search by ID or name, calculate a shortest path, run BFS with a destination reachability check, check connectivity, or load the built-in fictional sample into an empty graph. The source implements graph vertex removal as a graph-module operation, but there is no coordinated location/path deletion action in the menu.

Not implemented: GPS, real-time traffic, live map APIs, geographic coordinates, persistence, route preferences, road closures, hash-based lookup, sorting, and a graphical interface.

3. Software and Hardware Requirements

Category	Requirement / observed preparation environment
Language	C11; source uses standard C headers and C11-compatible constructs.
Compiler	C-capable compiler; verified with MinGW GCC 6.3.0 using -std=c11 -Wall -Wextra -Wpedantic -g.
Operating environment	Terminal/command prompt. Prepared and tested on Windows.
Runtime dependencies	No external runtime packages; the application uses the C standard library and its own graph, location, and queue modules.
Test tooling	The module driver is C. The supplied CLI regression script is PowerShell for Windows.
Hardware	Ordinary computer capable of running a C compiler and terminal. No GPS, network, graphics hardware, or special sensor is required.

The project does not declare minimum CPU, RAM, or disk figures. The fixed graph capacity is 100 locations; names are limited to 63 bytes after trimming.

4. Existing System and Proposed Approach

No survey of a deployed campus navigation system or comparison with a college process was supplied. This report makes no claims about a real existing system. Within this teaching project, location names, IDs, and distances are entered manually.

The software separates the location directory from the graph representation: the linked list stores IDs and labels, and the graph stores IDs and the pairwise distance matrix. The menu coordinates these modules, while graph algorithms operate on the matrix. This is a demonstrator of data structures and algorithms, not a replacement for campus signage or a live routing service.

5. Data Structures Used

Singly linked list of locations

Each `Location` node contains an integer ID, a fixed-size character array (64 bytes including the terminator), and a pointer to the next node. `LocationList` tracks head, tail, and count. Nodes are allocated with `malloc` and freed by `location_list_destroy`. Adding and searching scan the list linearly. Names are trimmed at both ends and compared without ASCII letter case significance.

The application adds a graph vertex, then its list node. If list insertion fails, it attempts to remove the just-added graph vertex. Sample-data initialization uses temporary list/graph values and assigns them to the live map only after all sample records and edges are ready.

Weighted undirected graph

The `Graph` contains a location-ID array of length 100, an integer 100 x 100 adjacency matrix, and a vertex count. A matrix value greater than zero is an edge distance in metres; zero means no edge. Undirected paths write the same weight to both symmetric cells. A linear ID-to-index scan maps public IDs to matrix positions, so IDs need not be consecutive.

The matrix uses O(V^2) space and reserves storage for at most 100 vertices. Matrix scans make Dijkstra and BFS O(V^2) in this implementation. Zero is reserved for no edge, so zero-length edges are rejected.

Circular queue

The queue stores integers in a dynamically allocated array with capacity, front, rear, and count fields. Enqueue and dequeue wrap indices modulo capacity and reject full/empty operations. BFS enqueues matrix indices, marks each vertex when enqueued, and releases queue storage when traversal completes.

6. Algorithms Implemented

Dijkstra shortest path

All accepted edge distances are positive. Dijkstra initializes distances to infinity except the source (zero). Each iteration scans all vertices for the unvisited finite-distance vertex with smallest tentative distance, then scans its adjacency-matrix row and relaxes positive-weight edges. A predecessor index is stored whenever a smaller total is found. The destination is reconstructed by following predecessors backwards and reversing the resulting sequence. The code checks addition overflow and reports a route distance beyond the supported int range.

With a matrix and linear minimum search, running time is O(V^2); distance, predecessor, reverse-route, and visited arrays use O(V) additional space. It minimizes stored distance, not time, safety, accessibility, or traffic.

Dijkstra pseudocode

```
DIJKSTRA(G, source, destination):
  for each v: dist[v] = infinity; pred[v] = NONE; visited[v] = false
  dist[source] = 0
  repeat at most V times:
    u = unvisited vertex with smallest finite dist[u] (linear scan)
    if no such u: stop
    if u is destination: stop
    mark u visited
    for each v in matrix row u:
      if weight[u][v] > 0 and v is unvisited:
        candidate = dist[u] + weight[u][v] (check overflow)
        if candidate < dist[v]: dist[v] = candidate; pred[v] = u
  if destination is infinite: report no route
  follow pred from destination to source; reverse collected IDs
  return route and distance, or capacity/overflow error
```

Breadth-first search and connectivity

BFS starts from a valid source and creates a queue with capacity equal to vertex count. It marks the source before enqueueing. It repeatedly dequeues a vertex, appends its ID to traversal order, and scans all matrix columns. Each unvisited positive-weight neighbor is marked and enqueued once. It returns reachable IDs in BFS order; it does not optimize distance. Connectivity runs BFS from the first vertex and compares the reached count with vertex count. An empty graph is not connected; a single-vertex graph is connected.

```
BFS(G, source):
  mark source visited; enqueue source index
  while queue is not empty:
    u = dequeue; append ID[u] to output
    for v from 0 to V-1:
      if weight[u][v] > 0 and v is not visited:
        mark v visited; enqueue v
  return output in dequeue order

CONNECTED(G):
  if V == 0: return false
  reachable = BFS(G, first vertex)
  return count(reachable) == V
```

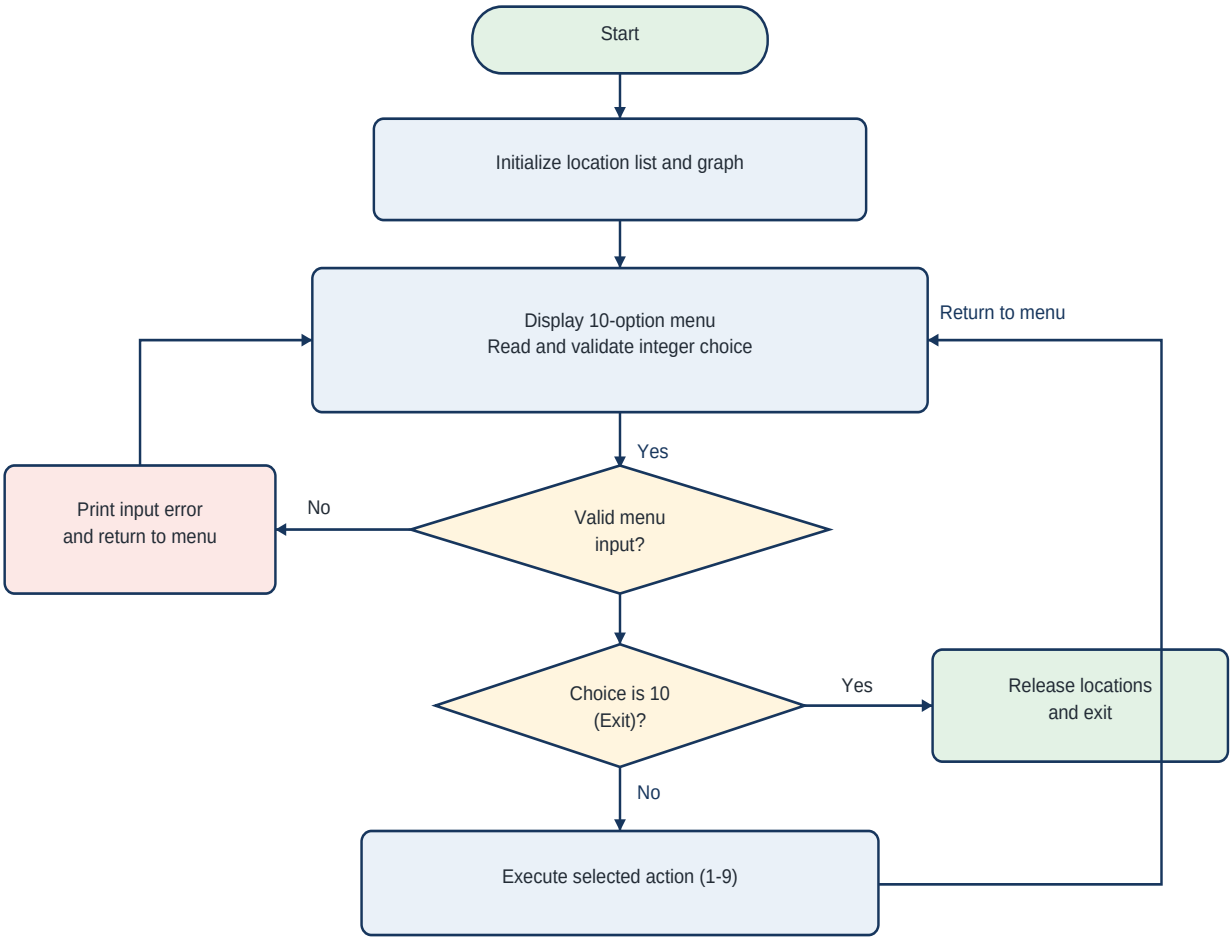
BFS uses O(V^2) time with the adjacency matrix and O(V) additional storage for visited/output/queue. Connectivity has the same asymptotic cost because it performs one BFS.

7. System Architecture and Program Flow

`main.c` owns the menu, validates input, coordinates modules, formats location references, and loads the optional demo graph. `locations.c` handles the linked-list directory. `graph.c` implements the matrix, path algorithms, and connectivity. `queue.c` provides FIFO storage to BFS. Header files define public types and operations. There is no database, network client, map service, or GUI layer.

Flowchart

Figure 1. Program workflow based on the implemented menu and exit behavior:



The menu loops after each completed action and after invalid integer input. Option 10 exits the loop; the location list is destroyed before normal termination. End-of-file input exits safely and reaches cleanup.

8. Module-wise Explanation and Implementation Details

File	Responsibility verified in source
main.c	10-option menu; bounded line and integer input; add/search/connect/display actions; Dijkstra and BFS presentation; connectivity; transactional fictional sample load; cleanup.
locations.h / locations.c	Location/list types and statuses; trimmed bounded names; duplicate ID/name checks; ID/name lookup; list display and destruction.
graph.h / graph.c	100-vertex weighted undirected graph; ID-index scans; vertex/edge operations; Dijkstra; BFS; connectivity; graph status text.
queue.h / queue.c	Dynamically allocated circular FIFO queue with full/empty checks, wraparound, and destruction.
test_driver.c	Separate assert-style C test executable for location, queue, and graph modules; defines its own main().
test_cli.ps1	Windows PowerShell interactive regressions for malformed/overlong input, duplicates, sample route, BFS, and EOF cases.

Input validation reads lines with `fgets()` and parses integers with `strtol()`. It checks range errors and rejects trailing non-whitespace text. Overlong lines are discarded and reported. IDs and distances must be positive; edges must join distinct existing vertices; duplicate undirected paths are rejected.

The graph and location list are separate structures synchronized by menu insertion. The sample loader builds temporary structures, then commits them together only when all five locations and seven edges have been created successfully. All state is in memory and is cleared at exit.

9. Input, Output, Compilation, and Execution

Menu options

Option	Action
1	Add campus location
2	Display all campus locations
3	Search by ID or name
4	Connect two existing locations with a distance
5	Display connected paths
6	Find a minimum-distance route using Dijkstra
7	List BFS-reachable locations and test destination reachability
8	Check graph connectivity
9	Load fictional sample data into an empty map
10	Exit

Integer inputs are decimal whole numbers. A location has a positive integer ID and nonblank name of up to 63 bytes after trimming. An edge requires two distinct existing IDs and a positive integer distance in metres. Output includes status messages, location/path listings, route IDs and names, total metres, BFS order, and reachability/connectivity findings.

Build commands

```
gcc -std=c11 -Wall -Wextra -Wpedantic -g main.c locations.c graph.c queue.c -o campus_navigation.exe
.\campus_navigation.exe

gcc -std=c11 -Wall -Wextra -Wpedantic -g test_driver.c locations.c graph.c queue.c -o test_driver.exe
.\test_driver.exe
```

The application and test driver must be compiled as separate executables because each has a main() function. The verified build used GCC 6.3.0 (MinGW) on Windows, with no external runtime dependency.

10. Testing Methodology and Results

Testing used a fresh compile from the current C source, not the pre-existing executables. The module test driver was compiled and run separately. The supplied PowerShell CLI script was run against the fresh application binary. It stopped at its third scenario on an expected-message mismatch, so a temporary copy was run with only the stale expected duplicate-ID wording corrected. Application source and the original test file were left unchanged.

Test / evidence	Inputs or check	Observed result
Fresh application build	C11 GCC; -Wall -Wextra -Wpedantic -g	PASS. GCC 6.3.0; no compiler diagnostics.
Module test driver	List, queue, graph assertions	PASS. Printed "Location, queue, and graph module tests passed."
Unmodified CLI case 1	Malformed, overflow, overlong menu lines and invalid menu values	PASS.
Unmodified CLI case 2	63-byte and overlong location names	PASS.
Unmodified CLI case 3	Duplicate ID/name insertion and rollback	Test assertion mismatch: expected "already in use"; app prints "already in the graph".
Adjusted temporary CLI copy	Only duplicate-ID expected message changed to observed application wording	PASS. All six named scenarios and all 13 EOF cases completed.
Fictional sample route	Load sample; route ID 10 to ID 900	PASS. Demo North Gate -> Demo Library -> Demo Cafeteria -> Demo Sports Centre; 510 m.
BFS	From ID 10; check destination 900	PASS. Order: 10, 73, 501, 420, 900.
Sample map display	Display five demo locations and seven edges	PASS. Actual terminal output is shown in Figures 2 and 5.

The CLI script mismatch is a test-fixture expected-string issue observed in this run, not evidence that the application emitted the expected phrase. The original script remains unchanged. No performance benchmark, real-campus validation, or external review is claimed.

11. Program Output Screenshots

The following images were captured from the compiled C program running in a Windows terminal. The labels and distances shown belong to the explicitly fictional demo data. Captions identify what each screen shows; no screenshot was synthesized.

Figure 2

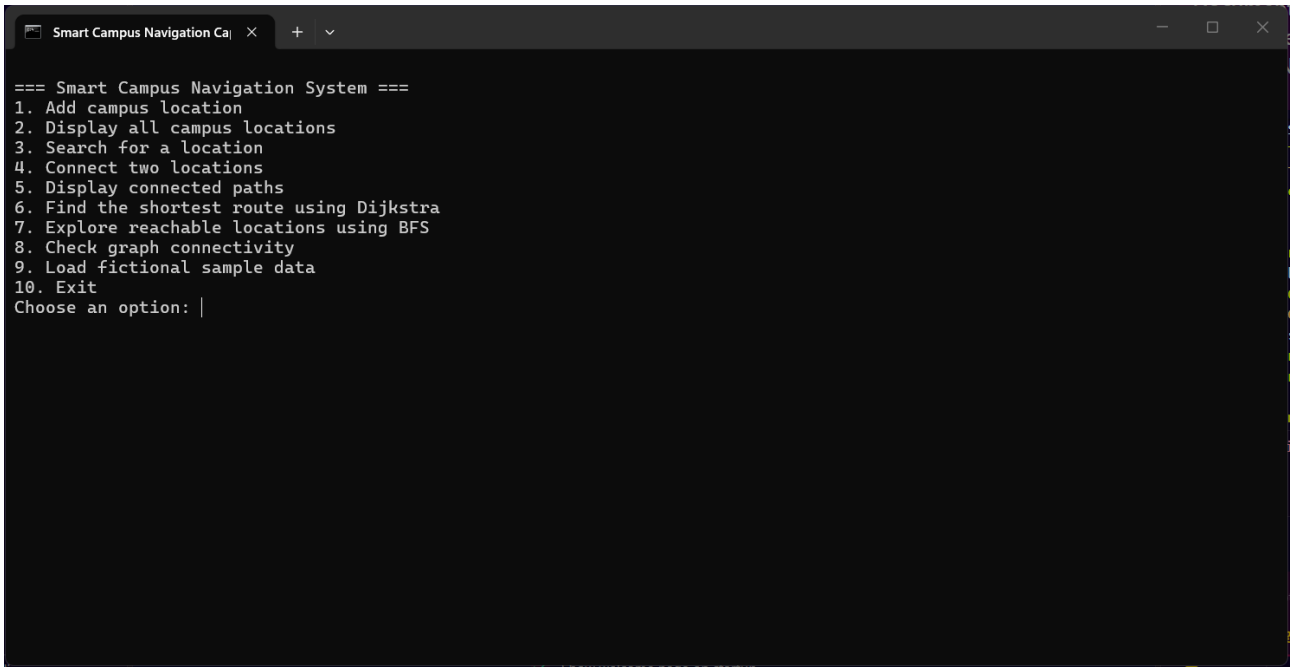


Figure 2. The compiled application launch menu. It lists the implemented location, path, Dijkstra, BFS, connectivity, demo-data, and exit actions.

Figure 3

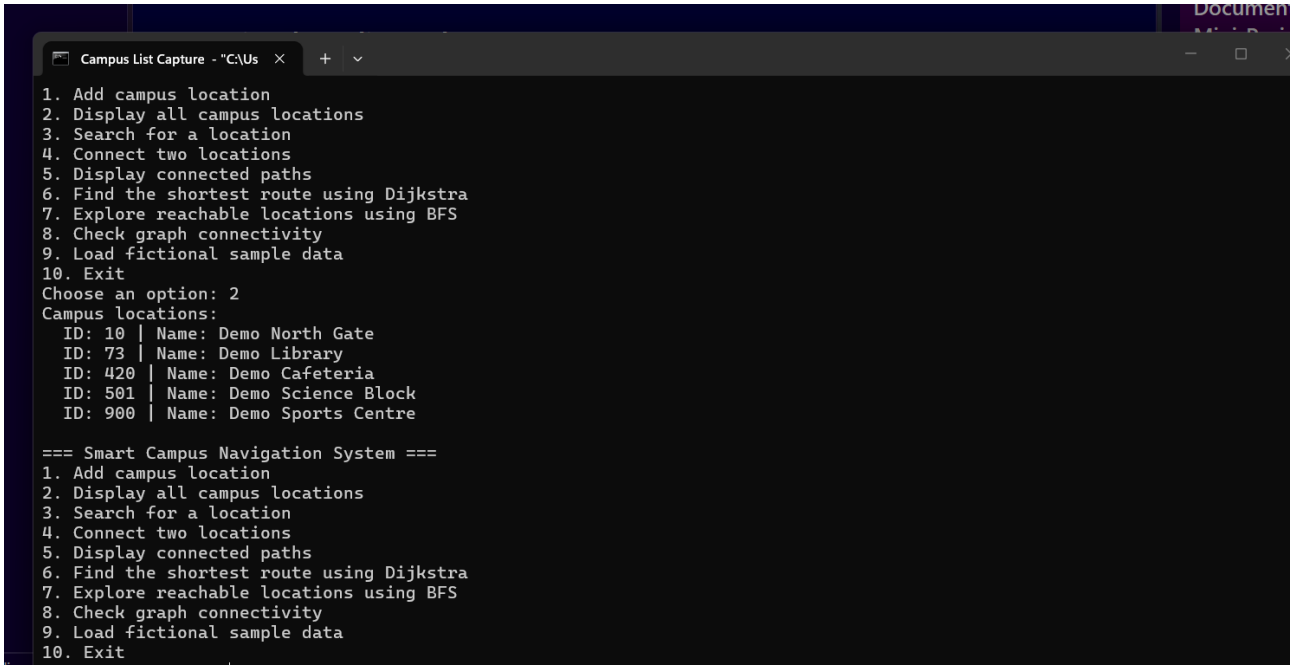


Figure 3. The explicitly fictional demonstration map loaded into the program and its five linked-list location records displayed.

Figure 4

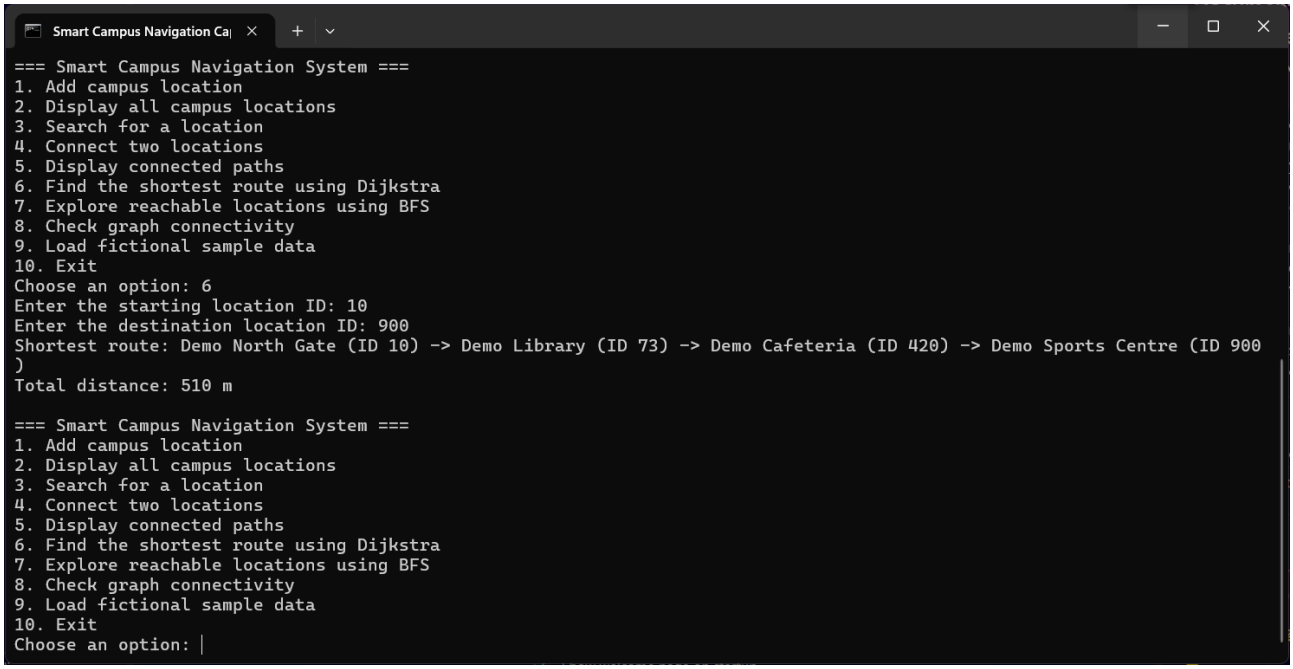


Figure 4. Dijkstra output for fictional sample IDs 10 to 900: 510 m via Demo Library and Demo Cafeteria.

The displayed 510 m total is printed by the application for the shown IDs and fictional sample network.

Figure 5

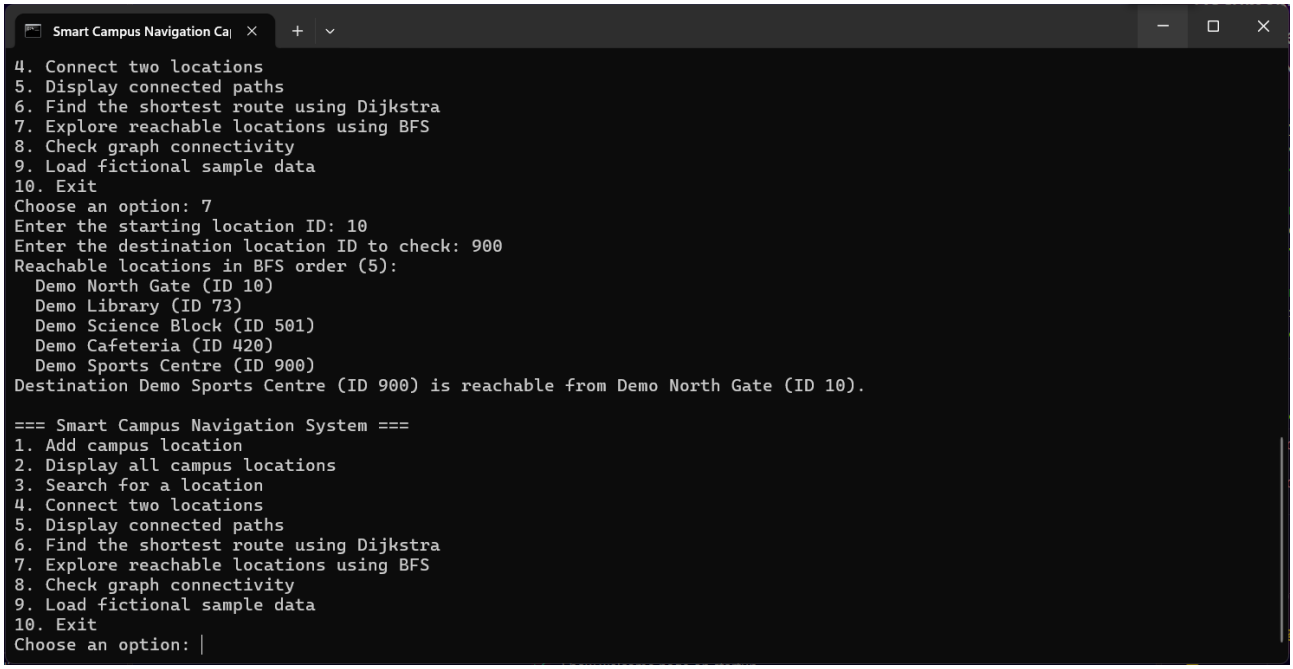


Figure 5. BFS from ID 10, showing the actual traversal order and reachability of destination ID 900.

Figure 6

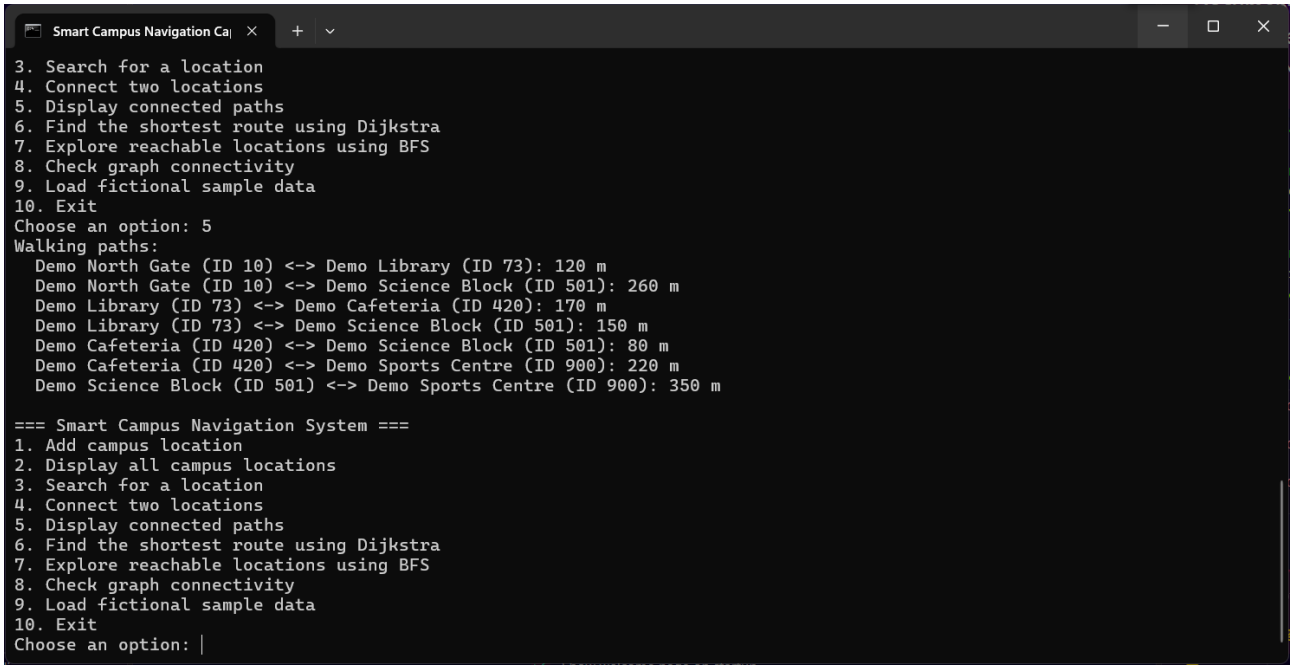


Figure 6. The seven undirected demonstration paths and their fictional distances in metres.

12. Limitations and Future Enhancements

Current limitations

- The adjacency matrix is fixed at 100 vertices and uses O(V^2) storage.
- All graph and location state is in memory and is lost when the program exits.
- Distances are manually entered positive integer metres and are not validated against a surveyed map.
- Sample location names and weights are fictional and do not describe a real campus.
- Name comparison handles ASCII case-insensitivity, not complete Unicode case folding.
- There is no menu operation for editing or deleting locations or paths; graph-module vertex removal is not a complete coordinated deletion workflow.
- The route optimizes total edge distance only. There are no coordinates, GPS, live traffic, accessibility metadata, or map visualization.
- No hash table, sorting, road-closure action, or alternative-route menu feature is implemented.

Possible future enhancements

Future work could add coordinated deletion/editing of locations and paths, optional validated file persistence, and import/export for verified campus data. A production navigation system would additionally require trustworthy mapped coordinates, route restrictions, accessibility and safety data, update procedures, and suitable validation; none is part of this implementation.

13. Conclusion

The Smart Campus Navigation System demonstrates modular C programming and core graph data structures and algorithms in a terminal application. A linked list manages named location records, an adjacency matrix represents positive undirected distances, a circular queue supports BFS, and a matrix-based Dijkstra implementation computes a minimum-distance route. The fictional sample yields a 510 m route from ID 10 to ID 900, and BFS enumerates the five sample locations in the actual order printed by the program.

A fresh warning-enabled build and the standalone module test driver passed. The supplied CLI regression script contains one duplicate-ID expected-string mismatch; all its scenarios passed in a temporary test copy after correcting only that expectation, without changing application source. This is a reproducible educational prototype, not a live or real-campus navigation service.

14. References

[1] Smart Campus Navigation System project source and README included with this submission: main.c, graph.c/.h, locations.c/.h, queue.c/.h, test_driver.c, README_Project.md. Primary reference for implemented behavior and sample output.

[2] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, pp. 269-271, 1959. DOI: 10.1007/BF01386390.

[3] ISO/IEC 9899:2018, Programming Languages - C. International Organization for Standardization, 2018. The project targets C11-compatible features and was compiled using -std=c11.

[4] GNU Project, GCC online documentation, https://gcc.gnu.org/onlinedocs/, for GCC command-line/toolchain context.

Personal details and signature fields remain placeholders for the student and authorized institution.